Sets in Python

10 August 2026 10:49

A set in Python is an unordered mutable collection of unique elements. Sets are particularly useful when you want to store items without duplicates and perform mathematical set operations such as union, intersection, difference, and symmetric difference.

What is a Set?

A set is a collection in which:

a. Elements are unique
b. Elements are unordered
c. A set of mutable meaningful elements can be added or removed.
d. Set elements must be hashable (for example Numbers, string, and tuples can be elements).
e. A set does not support indexing or slicing.

For example:

```
>>> numbers = {10, 20, 30, 40, 50}
>>> type(numbers)
<class 'set'>
>>> print(numbers)
{50, 20, 40, 10, 30}
```

The order is not guaranteed.

Creating a Set

A set can be created using curly braces {}.

An empty {} does not create a set. It creates a dictionary.

```
>>> n = {}
>>> n
{}
>>> type(n)
<class 'dict'>
```

To create an empty set, use:

```
>>> n = set()
>>> type(n)
<class 'set'>
```

Duplicate Elements

A major characteristics of a set is that a duplicate elements are automatically removed.

```
>>> n = {10, 20, 30, 10, 40, 20, 50}
>>> n
{50, 20, 40, 10, 30}
```

This makes it. Very useful for removing duplicates from data.

```
>>> li = [ 10, 10, 20, 30, 40, 10, 20, 40, 50]
>>> set1 = set(li)
>>> set1
{40, 10, 50, 20, 30}

>>> s = 'Hello'
>>> set2 = set(s)
>>> set2
{'e', 'l', 'o', 'H'}
>>> ucs = str(set2)
>>> ucs
"{'e', 'l', 'o', 'H'}"
>>>
```

Characteristics of Sets

```
>>> student = {'Arjun', 'Rahul', 'Shiv', 'Kritin'}
>>> student
{'Shiv', 'Rahul', 'Arjun', 'Kritin'}
>>> type(student)
<class 'set'>
```

A set has the following properties:

Property	Set
Ordered	No
Indexed	No
Allows duplicates	No
Mutable	Yes
Supports Slicing	No
Supports mathematical set operations	yes

Accessing Elements

Since sets are unordered, we cannot access elements using an index.

Instead, use a for loop.

```
>>> student
```

```
{'Shiv', 'Suveer', 'Arjun', 'Rahul', 'Kritin'}
>>> for e in student:
...     print(e)
...
Shiv
Suveer
Arjun
Rahul
Kritin
```

Adding elements

add()

The add() method adds one element to a set.

```
>>> student
{'Shiv', 'Rahul', 'Arjun', 'Kritin'}
>>> type(student)
<class 'set'>
>>> student.add('Suveer')
>>> student
{'Shiv', 'Suveer', 'Arjun', 'Rahul', 'Kritin'}
```

If we add an element that already exists, nothing happens.

```
>>> student.add('Shiv')
>>> for e in student:
...     print(e)
...
Shiv
Suveer
Arjun
Rahul
Kritin
```

Adding multiple elements

update()

The update () method adds multiple elements.

```
>>> student
{'Shiv', 'Suveer', 'Arjun', 'Rahul', 'Kritin'}

>>> student.update(['Amit', 'Sumit', 'Raja'])
>>> student
{'Shiv', 'Suveer', 'Sumit', 'Amit', 'Arjun', 'Raja', 'Rahul', 'Kritin'}
```

```
>>> numbers
{50, 20, 40, 10, 30}
>>> n
{50, 20, 40, 10, 30}
>>> m = {1, 4, 10, 20, 60}
>>> numbers.update(m)
>>> numbers
{1, 4, 40, 10, 50, 20, 60, 30}
```

Removing elements

Python provides several methods for removing elements from a set.

remove()

```
>>> numbers
{1, 4, 40, 10, 50, 20, 60, 30}
>>> numbers.remove(30)
>>> numbers
{1, 4, 40, 10, 50, 20, 60}

>>> numbers.remove(30)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    numbers.remove(30)
    ~~~~~~~~~~~~~~~^^^^
KeyError: 30
```

If the element does not exist, remove () raises a key error.

discard(): Also removes an element, but it does not produce an error if the element does not exist.

```
>>> numbers.discard(30)
>>> numbers
{1, 4, 40, 10, 50, 20, 60}
>>> numbers.discard(20)
>>> numbers
{1, 4, 40, 10, 50, 60}
>>>
```

pop()

The pop () method removes and returns an arbitrary elements from the set.

```
>>> numbers
{1, 4, 40, 10, 50, 60}
>>> x = numbers.pop()
>>> x
1
>>> x = numbers.pop()
>>> x
4
>>> numbers
{40, 10, 50, 60}
>>> x = numbers.pop()
>>> x
40
```

Because sets are unordered, you should not assume which element will be removed.

clear(): removes all elements

```
>>> numbers
{40, 10, 50, 60}
>>> numbers.clear()
>>> numbers
set()
```

The set still exists. It is simply empty.

Finding the number of elements

Use the len() function.

```
>>> n
{50, 20, 40, 10, 30}
>>> len(n)
5
```

Membership Testing

The **in** and **not in** operators can be used with sets.

```
>>> n
{50, 20, 40, 10, 30}
>>> len(n)
```

```
5
>>> 20 in n
True
>>> 5 in n
False
>>> 20 not in n
False
>>> 5 not in n
True
```

Sets are particularly efficient for membership test time.

Mathematical Operations on Sets

This is where sets become especially powerful.

Suppose:

```
>>> A = {1,2,3,4,5}
>>> B = {4, 5, 6, 7, 8}
>>> A
{1, 2, 3, 4, 5}
>>> B
{4, 5, 6, 7, 8}
>>>
```

We can perform:

i. Union
ii. Intersection
iii. Difference
iv. Symmetric Difference

Union

The union of two sets. Contains all elements from both the sets with duplicates removed.

A U B

Python:

```
>>> C = A | B
>>> C
{1, 2, 3, 4, 5, 6, 7, 8}
```

We can also use:

```
>>> C = A.union(B)
>>> C
{1, 2, 3, 4, 5, 6, 7, 8}
```

Intersection
The intersection contains elements common to both sets.
Mathematically:
A ∩ B

Python:

```
>>> A
{1, 2, 3, 4, 5}
>>> B
{4, 5, 6, 7, 8}
>>> C = A & B
>>> C
{4, 5}
>>> C = A.intersection(B)
>>> C
{4, 5}
```

Difference

The difference A - B contains elements that are present in A but not in B.

```
>>> A
{1, 2, 3, 4, 5}
>>> B
{4, 5, 6, 7, 8}
>>> C = A-B
>>> C
{1, 2, 3}
>>> D = B - A
>>> D
{8, 6, 7}
>>>
```

Symmetric Difference
Semantic difference contains elements that belongs to either set, but not both.
Mathematically:
A ^ B

```
>>> A
{1, 2, 3, 4, 5}
>>> B
{4, 5, 6, 7, 8}
>>> C = A ^ B
>>> C
```

```
{1, 2, 3, 6, 7, 8}
>>>

>>> C = A.symmetric_difference(B)
>>> C
{1, 2, 3, 6, 7, 8}
```

Subset: A set A is a subset of B if every element of A is present in B.

```
>>> A = {1, 2}
>>> B= {1,2,3,4}
>>> A.issubset(B)
True
>>> A <= B
True
```

Superset: A set B is a superset of A if it contains every element of A.

```
>>> A
{1, 2}
>>> B
{1, 2, 3, 4}
>>> B.issuperset(A)
True
>>> B >= A
True
>>> A >= B
False
```

Disjoint Sets
Two sets are disjoint if they have no elements in common.

```
>>> A = {1, 2, 3}
>>> B = {4, 5, 6}
>>> A.isdisjoint(B)
True
>>> A
{1, 2, 3}
>>> B = {4, 3, 5}
>>> B
{3, 4, 5}
>>> A.isdisjoint(B)
False
>>>
```

Frozen Set: Python also provides a frozen set. A frozen set is an immutable set.

```
>>> numbers = frozenset([10, 20, 30, 40, 50, 20])
>>> numbers
frozenset({40, 10, 50, 20, 30})
>>>
```

You cannot modify it.

Next class:

- List comprehension
- Set comprehension

Homework

Below are **5 long programming questions** designed to provide comprehensive practice on Python **Sets**, covering creation, traversal, `add()`, `update()`, `remove()`, `discard()`, union, intersection, difference, symmetric difference, subset, superset, disjoint sets, duplicate removal, and set comprehension.

Question 1 – Student Subject Analysis

Write a Python program to maintain the names of students enrolled in three different subjects: **Python, Java, and C++.** Store the student names in three separate sets.

The program should perform the following operations:

1. Display the names of all students enrolled in Python.
2. Display the names of students who are studying **both Python and Java.**
3. Display the names of students who are studying **Python but not Java.**
4. Display the names of students who are studying **Java but not C++.**
5. Display the names of students who are studying **at least one of the three subjects.**
6. Display the names of students who are studying **all three subjects.**
7. Display the names of students who are studying **exactly one subject.**
8. Check whether all Python students are also Java students using `issubset()`.
9. Check whether the Java student set is a superset of the Python student set.
10. Check whether the Python and C++ student sets are disjoint.

Use appropriate **set operators and set methods** wherever possible.

Question 2 – Remove Duplicates and Analyze Numbers

Write a Python program that accepts **20 integers** from the user and stores them in a list. The list may contain duplicate values.

Convert the list into a set and perform the following operations:

1. Display the original list.
2. Display the set containing only unique elements.
3. Display the number of unique elements.
4. Display the largest and smallest elements.
5. Create a set containing all **even numbers**.
6. Create a set containing all **odd numbers**.
7. Create another set containing the numbers divisible by **3**.
8. Display the numbers that are both even and divisible by 3.
9. Display the numbers that are odd but not divisible by 3.
10. Create a set containing the squares of all unique numbers using **set comprehension**.
11. Ask the user for a number and check whether it exists in the set using the `in` operator.
12. Demonstrate the use of `add()`, `discard()`, and `remove()` on the resulting set.

The program should clearly display the result of each operation.

Question 3 – Two Class Examination Analysis

A school conducts two examinations: **Computer Science** and **Mathematics**. Write a Python program that stores the roll numbers of students who passed each examination in two sets.

For example:

```
computer_science = {101, 102, 103, 105, 107, 110}
mathematics = {102, 103, 104, 107, 108, 110}
```

Your program should:

1. Display students who passed **both examinations**.
2. Display students who passed only Computer Science.
3. Display students who passed only Mathematics.
4. Display all students who passed at least one examination.
5. Display students who passed exactly one examination.
6. Find the total number of students who passed at least one examination.
7. Find the total number of students who passed both examinations.
8. Check whether the Computer Science set is a subset of the Mathematics set.
9. Check whether the Mathematics set is a superset of the Computer Science set.
10. Check whether two user-supplied sets of roll numbers are disjoint.
11. Add a newly passed student to the Computer Science set using `add()`.
12. Add several newly passed students using `update()`.
13. Remove a student using `remove()`.
14. Attempt to remove another roll number using `discard()` and observe the difference between `remove()` and `discard()`.

Use both **set operators and equivalent set methods** wherever appropriate.

Question 4 – Common and Unique Words in Two Sentences

Write a Python program that accepts **two sentences** from the user.

The program should use sets to perform the following operations:

1. Convert both sentences into sets of words.
2. Display all unique words appearing in the first sentence.
3. Display all unique words appearing in the second sentence.
4. Display the words common to both sentences.
5. Display the words occurring only in the first sentence.
6. Display the words occurring only in the second sentence.
7. Display all unique words occurring in either sentence.
8. Display the words occurring in exactly one of the two sentences using **symmetric difference**.
9. Display the number of unique words in each sentence.
10. Check whether every word in the first sentence also occurs in the second sentence.
11. Check whether the two sets of words are disjoint.
12. Create a set containing the lengths of all unique words using **set comprehension**.
13. Display all unique words having more than five characters using set comprehension.
14. Ask the user for a word and determine whether it occurs in either sentence.

Note: The program should treat uppercase and lowercase versions of the same word as identical. For example, "Python" and "python" should be considered the same word.

Question 5 – Shopping Cart and Product Analysis

Write a Python program to simulate a shopping system using sets.

Maintain three sets:

electronics
clothing
books

Each set should contain product IDs purchased by customers.

For example:

```
electronics = {101, 102, 103, 104, 105}
clothing = {103, 104, 106, 107}
books = {101, 104, 108, 109}
```

The program should perform the following operations:

1. Display all products purchased from each category.
2. Find products purchased from **both electronics and clothing.**
3. Find products purchased from **electronics and books.**
4. Find products purchased from **all three categories.**
5. Find products purchased only from electronics.
6. Find products purchased only from clothing.
7. Find products purchased only from books.
8. Find all unique products purchased from at least one category.
9. Find products purchased from exactly two categories.
10. Find products purchased from exactly one category.
11. Check whether all products in one category are also present in another category using `issubset()`.
12. Check whether one category is a superset of another.
13. Check whether two categories are disjoint.
14. Add a new product to a category using `add()`.
15. Add multiple new products using `update()`.
16. Remove a product using `remove()`.
17. Safely remove a product using `discard()`.
18. Create a set of product IDs greater than 105 using **set comprehension.**
19. Display the total number of unique products sold.
20. Display the product IDs that occur in more than one category.

Challenge: Solve the problem using **set operations rather than nested loops wherever possible.**